

VEDA Technology Internship — Task 7 Report

Basic Calculator Application Documentation

1. Project Overview

This report documents the implementation and verification of **Task 7 (Basic Calculator)** for the VEDA Technology Web Development Internship. The application delivers a clean, light, and responsive web calculator built strictly with HTML5, CSS3, and Vanilla JavaScript without external frameworks, libraries, or dynamic eval() calls.

2. Technical Architecture & Component Stack

Component	Implementation & Design Details
HTML5	Semantic structure (<header>, <main>, <section>), accessible keypad buttons with aria-label tags.
CSS3 Grid	CSS Grid 4-column layout (repeat(4, 1fr)), light slate theme (#f1f5f9 / #ffffff), subtle button active transform scaling, mobile responsive breakpoints.
Vanilla JS	Event delegation keypad listener, explicit state variables (currentInput, previousInput, operator), floating-point precision correction, zero eval() usage.

3. Keypad Layout & Operations Table

Operation	Symbol	Key Class / Column Span	Functionality Description
Clear	C	btn-clear (Spans 2 cols)	Resets all calculator state, displays, and operands to 0.
Delete	■	btn-delete (1 col)	Removes the last entered character/digit.
Division	÷	btn-operator (1 col)	Divides previous number by current number.
Multiplication	×	btn-operator (1 col)	Multiplies previous number by current number.
Subtraction	-	btn-operator (1 col)	Subtracts current number from previous number.
Addition	+	btn-operator (1 col)	Adds current number to previous number.
Zero Digit	0	btn-zero (Spans 2 cols)	Appends digit 0 (handles leading zero rule).
Decimal Point	.	btn-decimal (1 col)	Appends decimal (prevents duplicate decimal entries).
Equals	=	btn-equals (1 col)	Computes and displays the final result.

4. Input Validation & Safety Rules

- **Zero eval() Policy:** All arithmetic operations are executed using explicit JS arithmetic operators (+, -, *, /).
- **Divide-by-Zero Safety:** Catch division by 0 prior to evaluation and output *"Cannot divide by zero"* without throwing runtime exceptions or crashing the UI state.
- **Decimal Validation:** Strictly checks !currentInput.includes(".") before appending decimal point, preventing malformed values such as 2.5.3.
- **Floating-Point Precision:** Applies Math.round(result * 1e10) / 1e10 to eliminate IEEE 754 precision artifacts (e.g. 2.5 + 3.5 = 6).

5. Verification Matrix

Test Case	Input Sequence	Expected Output	Status
Addition	10 + 5 =	15	PASSED
Subtraction	10 - 5 =	5	PASSED
Multiplication	10 × 5 =	50	PASSED
Division	10 ÷ 5 =	2	PASSED
Decimal Math	2.5 + 3.5 =	6	PASSED

Divide by Zero	$5 \div 0 =$	Cannot divide by zero	PASSED
Clear Reset	Press C	Resets display to 0	PASSED
Multiple Decimals	$2 . 5 . 3$	2.53	PASSED
Chained Operations	$10 + 5 = + 5 =$	20	PASSED

6. Repository & Deployment Information

- **GitHub Repository:** <https://github.com/dharmarayank946/Basic-Calculator.git>
- **Local Host Server:** <http://localhost:8000>